Objetivos:

- Compreender e configurar permissões de acesso aos recursos do dispositivo.
- Consultar e apresentar contatos e números de telefone.
- Obter e exibir a localização atual do dispositivo.
- Capturar fotografias e selecionar imagens da galeria.
- Tratar recusas de autorização, cancelamentos e falhas, atualizando a interface após operações assíncronas.

Indice:

i. Permissões e acesso a recursos do dispositivo móvel
ii. Acessando contatos do dispositivo com Flutter
iii. Acessando a localização do dispositivo
iv. Acessando a câmera e a galeria do dispositivo

i. Permissões e acesso a recursos do dispositivo móvel

1.1. Objetivo

Compreender como um aplicativo Flutter solicita autorização para acessar recursos protegidos do dispositivo, como contatos, localização e câmera.

Ao final deste capítulo, você deverá ser capaz de:

- distinguir a configuração de uma permissão da sua solicitação durante a execução;
- identificar os arquivos de configuração do Android e do iOS;
- compreender o fluxo de verificação, solicitação e tratamento das permissões;
- planejar o comportamento do aplicativo quando o acesso não for autorizado.

1.2. Recursos do dispositivo móvel

Um aplicativo pode utilizar recursos do celular para oferecer funcionalidades que vão além da apresentação de telas e do armazenamento local de dados.

Considere os seguintes exemplos:

Recurso	Exemplo de utilização
Contatos	Consultar nomes e números de telefone da agenda.
Localização	Obter a posição atual para identificar locais próximos.
Câmera	Capturar uma fotografia para o perfil do usuário.
Galeria	Selecionar uma imagem para anexar a um cadastro.

Recurso	Exemplo de utilização
Microfone	Gravar uma mensagem de áudio.

Alguns desses recursos permitem o acesso a informações pessoais. Por esse motivo, o sistema operacional controla seu uso.

Adicionar um pacote ao projeto Flutter não significa que o aplicativo esteja autorizado a acessar o recurso. A integração precisa respeitar as regras do Android ou do iOS.

Também é necessário distinguir o acesso direto a um recurso da utilização de um seletor do sistema. Por exemplo, selecionar uma imagem pelo seletor de fotos do Android pode dispensar uma solicitação de permissão ampla para a galeria. A autorização necessária depende da operação realizada.

1.3. O que é uma permissão?

Uma **permissão** é uma autorização para que o aplicativo execute uma operação protegida pelo sistema operacional.

Por exemplo, um aplicativo que precisa ler a agenda deve verificar se possui autorização para acessar os contatos. Caso não possua, poderá solicitar essa autorização ao usuário.

A solicitação normalmente utiliza uma janela apresentada pelo próprio sistema operacional. O aplicativo inicia o pedido, mas o usuário decide se permite o acesso.

No Android, determinadas permissões precisam ser declaradas no projeto e solicitadas durante a execução. Portanto, **declarar uma permissão não equivale a obtê-la**.

Considere um aplicativo com três botões:

- Consultar contatos;
- Obter localização;
- Tirar fotografia.

Cada ação pode exigir uma autorização diferente. Permitir o acesso aos contatos não autoriza automaticamente o acesso à localização ou à câmera.

1.4. Configuração das permissões no projeto

Embora o código principal do aplicativo seja escrito em Dart, um projeto Flutter contém arquivos específicos para cada plataforma.

Esses arquivos permitem configurar como o aplicativo se integra ao sistema operacional.

1.4.1. Configuração no Android

No Android, as permissões são declaradas no arquivo:

`android/app/src/main/AndroidManifest.xml`

A declaração utiliza o elemento <uses-permission>, colocado dentro de <manifest> e fora de <application>.

Exemplo de declaração para leitura dos contatos:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.READ_CONTACTS" />

    <application
        android:label="aula7"
        android:name="${applicationName}"
        android:icon="@mipmap/ic_launcher">
        <!-- Configurações existentes do aplicativo.-->
    </application>
</manifest>
```

Essa declaração informa que o aplicativo pretende utilizar a permissão de leitura dos contatos.

Não substitua todo o arquivo pelo exemplo. Ao configurar o projeto, acrescente a declaração necessária e preserve as configurações existentes.

A declaração deve ser acompanhada pela verificação e, quando necessária, pela solicitação da autorização durante a execução.

1.5. Acesso aos recursos com Flutter

Para acessar recursos nativos, é comum utilizar **plugins**: pacotes que conectam o código Dart às funcionalidades das plataformas.

Em termos gerais, essa integração funciona da seguinte maneira:

1. O usuário aciona uma funcionalidade na interface.
2. O código Dart chama um método disponibilizado pelo plugin.
3. O plugin utiliza as APIs nativas do Android ou do iOS.
4. O resultado da operação retorna ao código Dart.
5. O aplicativo apresenta os dados ou uma mensagem ao usuário.

Existem plugins voltados ao gerenciamento de permissões e plugins específicos para contatos, localização e imagens. Dependendo do pacote, a solicitação de autorização pode estar integrada à própria operação.

Por isso, antes de implementar uma funcionalidade, é necessário consultar a documentação do plugin para identificar:

- as plataformas suportadas;
- as configurações exigidas;
- os métodos de verificação e solicitação de acesso;
- os resultados e erros que precisam ser tratados.

1.6. Fluxo de solicitação de uma permissão

O aplicativo deve verificar a autorização antes de realizar uma operação protegida.

O fluxo básico é:

1. O usuário solicita uma funcionalidade.
2. O aplicativo verifica a permissão.
3. Se estiver autorizada, tenta executar a operação.
4. Se não estiver autorizada e uma solicitação for possível, solicita o acesso.
5. O aplicativo analisa a resposta e apresenta o resultado apropriado.

O Android orienta que a solicitação esteja relacionada à ação do usuário e que a interface continue utilizável quando a permissão for negada.

O pseudocódigo a seguir representa essa lógica. **Ele é ilustrativo e não utiliza a API de um pacote específico.**

```
quando o usuário pressionar "Consultar contatos":
    verificar a autorização
    se o acesso estiver autorizado:
        tentar consultar os contatos
    senão, se uma solicitação for possível:
        solicitar autorização
        se o usuário autorizar:
            tentar consultar os contatos
        senão:
            informar que o acesso não foi autorizado
    senão:
        explicar a situação
        oferecer acesso às configurações, quando adequado
```

Perceba que a consulta acontece somente após a autorização. Mesmo assim, a operação ainda pode falhar e precisa de tratamento de erros.

1.7. Permissão, disponibilidade e resultado da operação

Ter autorização é apenas uma das condições necessárias para utilizar uma funcionalidade.

Considere os exemplos:

Funcionalidade	Permissão	Outra condição a verificar
Consultar contatos	Acesso autorizado	Existência de contatos e de números cadastrados.
Obter localização	Acesso autorizado	Serviço de localização ativo e obtenção de uma posição.

Funcionalidade	Permissão	Outra condição a verificar
Capturar fotografia	Acesso permitido conforme a integração	Disponibilidade da câmera e conclusão da captura.
Selecionar imagem	Acesso conforme o seletor utilizado	Seleção efetiva de uma imagem pelo usuário.

Por exemplo, o usuário pode abrir a câmera e retornar sem tirar uma fotografia. Também pode abrir a galeria e cancelar a seleção.

Cancelamento, recusa de permissão e falha técnica são situações diferentes. O aplicativo deve identificá-las para apresentar mensagens apropriadas.

As operações também são normalmente assíncronas. Enquanto aguarda uma resposta do sistema ou do usuário, o aplicativo pode apresentar um indicador de carregamento e impedir solicitações duplicadas.

ii. Acessando contatos do dispositivo com Flutter

2.1. Objetivo

Desenvolver um aplicativo Flutter que solicite autorização para ler os contatos do dispositivo e apresente seus nomes e números de telefone.

O exemplo permitirá:

- verificar e solicitar a permissão de leitura;
- carregar os contatos de forma assíncrona;
- apresentar os contatos em uma lista;
- selecionar um contato para visualizar seus telefones;
- tratar permissão negada, lista vazia e falhas na consulta.

2.2. Preparação do projeto

Utilizaremos o pacote `flutter_contacts`, que disponibiliza operações de acesso à agenda e métodos de gerenciamento das permissões.

Este capítulo utiliza a API da versão 2, na qual a consulta é realizada com `FlutterContacts.getAll()` e as permissões são gerenciadas por `FlutterContacts.permissions`.

Crie o projeto e adicione a dependência:

```
flutter create app_nome --platforms=android
```

Adicione o pacote:

```
flutter pub add flutter_contacts
```

2.3. Configuração das plataformas

Abra:

`android/app/src/main/AndroidManifest.xml`

Adicione a permissão dentro de `<manifest>`, antes de `<application>`:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.READ_CONTACTS" />

    <application
        android:label="aula7"
        android:name="${applicationName}"
        android:icon="@mipmap/ic_launcher">
        <!-- Configurações existentes do aplicativo.-->
    </application>
</manifest>
```

Neste exemplo, solicitaremos somente leitura, utilizando `PermissionType.read`. O pacote distingue permissões de leitura, escrita e leitura com escrita.

Se encontrar problemas durante a compilação, adicione as variáveis no arquivo `android/gradle.properties`:

```
kotlin.compiler.execution.strategy=in-process
kotlin.incremental=false
```

2.4. Código completo

Cole o código a seguir no arquivo `lib/main.dart`:

```
import 'package:flutter/material.dart';
import 'package:flutter_contacts/flutter_contacts.dart';

void main() {
  runApp(
    const MaterialApp(debugShowCheckedModeBanner: false, home: TelaContatos()),
  );
}

class TelaContatos extends StatefulWidget {
  const TelaContatos({super.key});

  @override
  State<TelaContatos> createState() => _TelaContatosState();
}

class _TelaContatosState extends State<TelaContatos> {
  List<Contact> _contatos = [];

  bool _carregando = false;
  bool _mostrarConfiguracoes = false;
```

```
String _mensagem = 'Pressione o botão para consultar os contatos.';

String _nomeDoContato(Contact contato) {
  final nome = contato.displayName;

  if (nome == null || nome.trim().isEmpty) {
    return 'Contato sem nome';
  }

  return nome;
}

String _telefonesDoContato(Contact contato) {
  if (contato.phones.isEmpty) {
    return 'Telefone não cadastrado';
  }

  return contato.phones.map((telefone) => telefone.number).join('\n');
}

Future<void> _carregarContatos() async {
  if (_carregando) return;

  setState(() {
    _carregando = true;
    _mostrarConfiguracoes = false;
    _contatos = [];
    _mensagem = 'Verificando acesso aos contatos...';
  });

  try {
    var permissao = await FlutterContacts.permissions.check(
      PermissionType.read,
    );

    if (!mounted) return;

    if (permissao == PermissionStatus.notDetermined ||
        permissao == PermissionStatus.denied) {
      permissao = await FlutterContacts.permissions.request(
        PermissionType.read,
      );
    }

    if (!mounted) return;

    final autorizado =
        permissao == PermissionStatus.granted ||
```

```
        permissao == PermissionStatus.limited;

    if (!autorizado) {
      setState(() {
        if (permissao == PermissionStatus.permanentlyDenied) {
          _mensagem =
              'Autorize o acesso aos contatos nas configurações '
              'do aplicativo.';
          _mostrarConfiguracoes = true;
        } else if (permissao == PermissionStatus.restricted) {
          _mensagem = 'O acesso aos contatos está restrito pelo sistema.';
        } else {
          _mensagem = 'O acesso aos contatos não foi autorizado.';
        }
      });

      return;
    }

    final contatos = await FlutterContacts.getAll(
      properties: {ContactProperty.phone},
    );

    if (!mounted) return;

    contatos.sort(
      (a, b) =>
          _nomeDoContato(a)
              .toLowerCase()
              .compareTo(_nomeDoContato(b).toLowerCase()),
    );

    setState(() {
      _contatos = contatos;

      if (contatos.isEmpty) {
        _mensagem = 'Nenhum contato disponível para consulta.';
      } else if (permissao == PermissionStatus.limited) {
        _mensagem =
            '${contatos.length} contato(s) disponível(is). '
            'O acesso está limitado aos contatos autorizados.';
      } else {
        _mensagem = '${contatos.length} contato(s) encontrado(s).';
      }
    });
  } catch (erro) {
    if (!mounted) return;

    setState(() {
```

```
        _mensagem = 'Não foi possível consultar os contatos.';
      });
    } finally {
      if (mounted) {
        setState(() {
          _carregando = false;
        });
      }
    }
  }

  Future<void> _abrirConfiguracoes() async {
    try {
      await FlutterContacts.permissions.openSettings();
    } catch (erro) {
      if (!mounted) return;

      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text('Não foi possível abrir as configurações.'),
        ),
      );
    }
  }

  void _mostrarContato(Contact contato) {
    showDialog<void>(
      context: context,
      builder: (context) {
        return AlertDialog(
          title: Text(_nomeDoContato(contato)),
          content: Text(_telefonesDoContato(contato)),
          actions: [
            TextButton(
              onPressed: () => Navigator.pop(context),
              child: const Text('Fechar'),
            ),
          ],
        );
      },
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Contatos do dispositivo')),
      body: Padding(
        padding: const EdgeInsets.all(16),
```

```
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            ElevatedButton(
              onPressed: _carregando ? null : _carregarContatos,
              child: const Text('Consultar contatos'),
            ),
            const SizedBox(height: 12),
            Text(_mensagem),
            if (_mostrarConfiguracoes) ...[
              const SizedBox(height: 8),
              OutlinedButton(
                onPressed: _abrirConfiguracoes,
                child: const Text('Abrir configurações'),
              ),
            ],
            const SizedBox(height: 16),
            Expanded(
              child: _carregando
                  ? const Center(child: CircularProgressIndicator())
                  : ListView.builder(
                      itemCount: _contatos.length,
                      itemBuilder: (context, index) {
                        final contato = _contatos[index];

                        return ListTile(
                          leading: const Icon(Icons.person),
                          title: Text(_nomeDoContato(contato)),
                          subtitle: Text(
                            contato.phones.isEmpty
                                ? 'Telefone não cadastrado'
                                : contato.phones.first.number,
                          ),
                          trailing: const Icon(Icons.chevron_right),
                          onTap: () => _mostrarContato(contato),
                        );
                      },
                    ),
            ),
          ],
        ),
      ),
    );
  }
}
```

O exemplo utiliza os métodos documentados de verificação, solicitação e abertura das configurações do aplicativo. A consulta inclui explicitamente os telefones, pois, por padrão, `getAll()` retorna apenas identificadores e nomes de exibição.

2.5. Organização do estado da tela

A tela utiliza `StatefulWidget` porque seu conteúdo muda durante a execução.

As principais variáveis são:

```
List<Contact> _contatos = [];

bool _carregando = false;
bool _mostrarConfiguracoes = false;

String _mensagem = 'Pressione o botão para consultar os contatos.';
```

Variável	Finalidade
`_contatos`	Armazena os contatos obtidos na consulta.
`_carregando`	Indica que uma operação está em andamento.
`_mostrarConfiguracoes`	Controla a apresentação do botão de configurações.
`_mensagem`	Apresenta orientações e resultados ao usuário.

A lista utiliza o tipo `Contact`, fornecido pelo pacote. Entre suas propriedades estão `displayName`, que pode ser nulo, e `phones`, uma lista de telefones.

As mudanças nessas variáveis são realizadas dentro de `setState()`, provocando a reconstrução da interface.

2.6. Verificação e solicitação da permissão

A verificação acontece quando o usuário pressiona **"Consultar contatos"**:

```
var permissao = await FlutterContacts.permissions.check( PermissionType.read );
```

A palavra-chave `await` aguarda a conclusão da operação assíncrona. O resultado informa o estado atual da autorização.

Se a permissão ainda não foi solicitada ou foi negada com possibilidade de nova solicitação, o código executa:

```
if(permissao == PermissionStatus.notDetermined || permissao == PermissionStatus.denied){
  permissao = await FlutterContacts.permissions.request( PermissionType.read );
}
```

Em seguida, verifica se a consulta pode continuar:

```
final autorizado = permissao == PermissionStatus.granted ||
                   permissao == PermissionStatus.limited;
```

O estado `limited` representa acesso parcial aos contatos no iOS. Nesse caso, o aplicativo deve trabalhar com os contatos disponibilizados pelo usuário. Já `permanentlyDenied` indica que a autorização precisa ser alterada nas configurações, e `restricted` representa uma restrição imposta pelo sistema.

A consulta não é executada quando o acesso não está autorizado.

2.7. Leitura dos contatos

Após obter autorização, o aplicativo consulta os contatos:

```
final contatos = await FlutterContacts.getAll(
  properties: {ContactProperty.phone},
);
```

O parâmetro `properties` informa quais dados adicionais devem ser carregados. Neste exemplo, precisamos dos números de telefone. O nome de exibição já acompanha a consulta.

iii. Acessando a localização do dispositivo

3.1. Objetivo

Desenvolver um aplicativo Flutter que obtenha a localização atual do dispositivo e apresente suas coordenadas geográficas.

O exemplo aborda:

- configuração das permissões de localização;
- verificação do serviço de localização;
- solicitação de autorização ao usuário;
- obtenção da latitude e da longitude;
- tratamento de permissão negada, serviço desativado e falhas na consulta.

3.2. Permissão e serviço de localização

Antes de consultar a posição, precisamos verificar duas condições diferentes:

1. O serviço de localização está ativado?
2. O aplicativo possui autorização para acessar a localização?

Um celular pode estar com a localização ativada, mas o aplicativo não possuir permissão. Também pode ocorrer o contrário: o aplicativo está autorizado, mas o serviço está desativado.

O pacote utilizado oferece métodos separados para verificar essas condições.

Situação	Comportamento do aplicativo
Serviço ativado e acesso autorizado	Tentar obter a posição.
Serviço desativado	Orientar o usuário a ativá-lo.
Acesso ainda não autorizado	Solicitar a permissão, quando possível.
Permissão negada	Informar que a consulta depende da autorização.
Permissão bloqueada	Orientar o usuário a revisar as configurações do aplicativo.

Mesmo com o serviço ativado e a permissão concedida, a obtenção da posição pode demorar ou falhar.

3.3. Instalação do pacote

Adicione o pacote:

`flutter pub add geolocator`

O pacote permite consultar a posição atual, verificar permissões e encaminhar o usuário às configurações.

Para reproduzir o exemplo a seguir, utilize um dispositivo ou emulador Android. A versão atual do pacote exige

3.4. Configuração da plataforma Android

Abra:

`android/app/src/main/AndroidManifest.xml`

Adicione as declarações dentro de `<manifest>` e fora de `<application>`:

```
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```

Essas permissões permitem solicitar localização aproximada e precisa. Mesmo quando o aplicativo solicita localização precisa, o usuário pode autorizar somente a aproximada.

Preserve o restante do arquivo. Caso esteja utilizando o projeto do capítulo anterior, mantenha também a declaração de leitura dos contatos.

3.5. Código completo do exemplo

Cole o código a seguir no arquivo `lib/main.dart`:

```
import 'dart:async';

import 'package:flutter/material.dart';
import 'package:geolocator/geolocator.dart';

void main() {
  runApp(
    const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: TelaLocalizacao(),
    ),
  );
}

class TelaLocalizacao extends StatefulWidget {
  const TelaLocalizacao({super.key});

  @override
  State<TelaLocalizacao> createState() => _TelaLocalizacaoState();
}

class _TelaLocalizacaoState extends State<TelaLocalizacao> {
```

```
Position? _posicao;

bool _carregando = false;
bool _mostrarConfiguracoesApp = false;
bool _mostrarConfiguracoesLocalizacao = false;

String _mensagem = 'Pressione o botão para obter a localização atual.';

Future<void> _obterLocalizacao() async {
  if (_carregando) return;

  setState(() {
    _carregando = true;
    _posicao = null;
    _mostrarConfiguracoesApp = false;
    _mostrarConfiguracoesLocalizacao = false;
    _mensagem = 'Verificando o serviço de localização...';
  });

  try {
    final servicoAtivo = await Geolocator.isLocationServiceEnabled();

    if (!mounted) return;

    if (!servicoAtivo) {
      setState(() {
        _mensagem = 'Ative o serviço de localização do dispositivo.';
        _mostrarConfiguracoesLocalizacao = true;
      });

      return;
    }

    var permissao = await Geolocator.checkPermission();

    if (!mounted) return;

    if (permissao == LocationPermission.denied) {
      permissao = await Geolocator.requestPermission();
    }

    if (!mounted) return;

    if (permissao == LocationPermission.deniedForever) {
      setState(() {
        _mensagem =
            'Autorize a localização nas configurações '
            'do aplicativo.';
        _mostrarConfiguracoesApp = true;
```

```
      });

      return;
    }

    final autorizado =
        permissao == LocationPermission.whileInUse ||
        permissao == LocationPermission.always;

    if (!autorizado) {
      setState(() {
        _mensagem = 'O acesso à localização não foi autorizado.';
      });

      return;
    }

    setState(() {
      _mensagem = 'Obtendo a posição atual...';
    });

    const configuracao = LocationSettings(
      accuracy: LocationAccuracy.high,
      timeLimit: Duration(seconds: 20),
    );

    final posicao = await Geolocator.getCurrentPosition(
      locationSettings: configuracao,
    );

    if (!mounted) return;

    setState(() {
      _posicao = posicao;
      _mensagem = 'Localização obtida com sucesso.';
    });
  } on TimeoutException {
    if (!mounted) return;

    setState(() {
      _mensagem =
          'Não foi possível obter a posição no tempo '
          'previsto. Tente novamente.';
    });
  } catch (erro) {
    if (!mounted) return;

    setState(() {
      _mensagem = 'Não foi possível obter a localização.';
```

```
      });
    } finally {
      if (mounted) {
        setState(() {
          _carregando = false;
        });
      }
    }
  }

  Future<void> _abrirConfiguracoes({required bool doAplicativo}) async {
    try {
      final abriu = doAplicativo
          ? await Geolocator.openAppSettings()
          : await Geolocator.openLocationSettings();

      if (!mounted) return;

      if (!abriu) {
        _informarFalhaNasConfiguracoes();
      }
    } catch (erro) {
      if (!mounted) return;

      _informarFalhaNasConfiguracoes();
    }
  }

  void _informarFalhaNasConfiguracoes() {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Não foi possível abrir as configurações.')),
    );
  }

  @override
  Widget build(BuildContext context) {
    final posicao = _posicao;

    return Scaffold(
      appBar: AppBar(title: const Text('Localização do dispositivo')),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            ElevatedButton(
              onPressed: _carregando ? null : _obterLocalizacao,
              child: const Text('Obter localização'),
            ),
```

```
const SizedBox(height: 16),
Text(_mensagem),
if (_mostrarConfiguracoesLocalizacao) ...[
  const SizedBox(height: 12),
  OutlinedButton(
    onPressed: () => _abrirConfiguracoes(doAplicativo: false),
    child: const Text('Configurações de localização'),
  ),
],
if (_mostrarConfiguracoesApp) ...[
  const SizedBox(height: 12),
  OutlinedButton(
    onPressed: () => _abrirConfiguracoes(doAplicativo: true),
    child: const Text('Configurações do aplicativo'),
  ),
],
if (_carregando) ...[
  const SizedBox(height: 24),
  const Center(child: CircularProgressIndicator()),
],
if (posicao != null) ...[
  const SizedBox(height: 24),
  Card(
    child: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            'Latitude: '
            '${posicao.latitude.toStringAsFixed(6)}',
          ),
          const SizedBox(height: 8),
          Text(
            'Longitude: '
            '${posicao.longitude.toStringAsFixed(6)}',
          ),
          const SizedBox(height: 8),
          Text(
            'Precisão horizontal estimada: '
            '${posicao.accuracy.toStringAsFixed(1)} m',
          ),
          const SizedBox(height: 8),
          Text(
            'Instante da posição: '
            '${posicao.timestamp.toLocal()}',
          ),
        ],
      ),
```

```
                ),
              ),
            ],
          ],
        ),
      ),
    );
  }
}
```

3.6. Armazenamento da posição no estado

A posição é armazenada nesta variável:

```
Position? _posicao;
```

O tipo `Position` representa o resultado da consulta e contém informações como latitude, longitude, precisão horizontal estimada e instante da posição.

O símbolo `?` permite que a variável contenha `null`. Isso é necessário porque, inicialmente, nenhuma localização foi obtida.

Ao iniciar uma nova consulta, o exemplo executa:

```
_posicao = null;
```

Assim, as coordenadas anteriores deixam de aparecer enquanto uma nova posição está sendo obtida.

Depois da consulta:

```
setState(() {
  _posicao = posicao;
  _mensagem = 'Localização obtida com sucesso.';
});
```

O `setState()` atualiza a interface para apresentar o resultado.

3.7. Verificação do serviço de localização

A primeira verificação utiliza:

```
final servicoAtivo = await Geolocator.isLocationServiceEnabled();
```

O resultado é um `bool`:

- `true`: serviço ativado;
- `false`: serviço desativado.

Se estiver desativado, o código apresenta uma orientação e encerra a consulta:

```
if (!servicoAtivo) {
  setState(() {
    _mensagem = 'Ative o serviço de localização do dispositivo.';
    _mostrarConfiguracoesLocalizacao = true;
  });
```

```
  return;
}
```

O `return` impede que o aplicativo continue tentando obter a posição nessa condição.

3.8. Verificação e solicitação da permissão

Com o serviço ativo, o aplicativo verifica a autorização:

```
var permissao = await Geolocator.checkPermission();
```

Se o resultado indicar acesso negado, solicita a permissão:

```
if (permissao == LocationPermission.denied) {
  permissao = await Geolocator.requestPermission();
}
```

Para permitir a consulta, o exemplo aceita:

```
final autorizado =
    permissao == LocationPermission.whileInUse ||
    permissao == LocationPermission.always;
```

`whileInUse` indica autorização durante o uso do aplicativo. `always` também é aceito caso já tenha sido concedido, mas o exemplo não solicita acesso em segundo plano.

Se a autorização estiver bloqueada, o aplicativo apresenta o botão para abrir suas configurações.

3.9. Obtenção da posição atual

A consulta utiliza:

```
const configuracao = LocationSettings(
  accuracy: LocationAccuracy.high,
  timeLimit: Duration(seconds: 20),
);

final posicao = await Geolocator.getCurrentPosition(
  locationSettings: configuracao,
);
```

A classe `LocationSettings` permite definir a precisão desejada e um limite de espera.

Neste exemplo:

Configuração	Finalidade
`LocationAccuracy.high`	Solicitar uma posição com maior precisão.
`Duration(seconds: 20)`	Limitar a espera pela obtenção da posição.

A precisão solicitada não é uma garantia de precisão obtida. Ela depende da autorização concedida e das condições de localização.

O limite de 20 segundos foi escolhido para o exemplo e pode ser ajustado. Se for excedido, o pacote lança uma `TimeoutException`, tratada separadamente no código.

O método retorna uma posição. Para obter outra, o usuário deve pressionar novamente o botão.

3.10. Acesso às configurações

O exemplo diferencia dois encaminhamentos:

```
Geolocator.openAppSettings()
```

Utilizado quando o usuário precisa revisar a autorização do aplicativo.

```
await Geolocator.openLocationSettings()
```

Utilizado quando o serviço de localização está desativado.

No Android, esses métodos encaminham para configurações distintas. No iOS, há limitações para abrir páginas específicas, e o usuário pode precisar navegar até a opção correspondente.

Ao retornar das configurações, pressione “Obter localização” novamente. O exemplo verifica o serviço e a permissão em cada consulta.

iv. Acessando a câmera e a galeria do dispositivo

4.1. Objetivo

Desenvolver um aplicativo Flutter que permita capturar uma fotografia pela câmera ou selecionar uma imagem da galeria e apresentá-la na interface.

Neste exemplo serão abordados:

- configuração do acesso à câmera e às imagens;
- captura de uma fotografia;
- seleção de uma imagem;
- apresentação da imagem na tela;
- tratamento de cancelamento e falhas;
- diferença entre selecionar uma imagem e armazená-la permanentemente.

4.2. Instalação do pacote

Adicione o pacote:

```
flutter pub add image_picker
```

Esse pacote abre interfaces nativas de captura e seleção, sem exigir a construção de uma interface própria para controlar a câmera.

4.3. Configuração da plataforma Android

Para o uso básico do `image_picker`, a documentação atual informa que não é necessário acrescentar configurações de permissões no `AndroidManifest.xml`.

Portanto, neste exemplo, não adicionaremos permissões de armazenamento ou de câmera manualmente. O pacote utiliza as interfaces do sistema para realizar as operações. Essa orientação é específica para essa integração; outros pacotes que controlam diretamente a câmera podem exigir configurações diferentes.

Caso esteja utilizando o projeto dos capítulos anteriores, preserve as permissões de contatos e localização já declaradas.

4.4. Código completo do exemplo

Cole o código a seguir no arquivo `lib/main.dart`:

```
import 'dart:io';

import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:image_picker/image_picker.dart';

void main() {
  runApp(
    MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Platform.isAndroid
          ? const TelaImagens()
          : const Scaffold(
              body: Center(
                child: Text('Este exemplo está disponível apenas no Android.'),
              ),
            ),
    ),
  );
}

class TelaImagens extends StatefulWidget {
  const TelaImagens({super.key});

  @override
  State<TelaImagens> createState() => _TelaImagensState();
}

class _TelaImagensState extends State<TelaImagens> {
  final ImagePicker _seletor = ImagePicker();

  XFile? _imagem;

  bool _carregando = false;
```

```
String _mensagem = 'Tire uma fotografia ou selecione uma imagem da galeria.';

@override
void initState() {
  super.initState();

  _recuperarImagemPendente();
}

Future<void> _recuperarImagemPendente() async {
  setState(() {
    _carregando = true;
  });

  try {
    final resposta = await _seletor.retrieveLostData();

    if (!mounted) return;

    if (resposta.isEmpty) return;

    final arquivos = resposta.files;

    setState(() {
      if (arquivos != null && arquivos.isNotEmpty) {
        _imagem = arquivos.first;
        _mensagem = 'Imagem recuperada após reiniciar o aplicativo.';
      } else {
        _mensagem = 'Não foi possível recuperar a imagem pendente.';
      }
    });
  } catch (erro) {
    if (!mounted) return;

    setState(() {
      _mensagem = 'Não foi possível verificar imagens pendentes.';
    });
  } finally {
    if (mounted) {
      setState(() {
        _carregando = false;
      });
    }
  }
}

Future<void> _obterImagem(ImageSource origem) async {
  if (_carregando) return;
```

```
setState(() {
  _carregando = true;
  _mensagem = origem == ImageSource.camera
      ? 'Aguardando a captura da fotografia...'
      : 'Aguardando a seleção da imagem...';
});

try {
  final imagem = await _seletor.pickImage(
    source: origem,
    maxWidth: 1200,
    maxHeight: 1200,
    imageQuality: 85,
    requestFullMetadata: false,
  );

  if (!mounted) return;

  if (imagem == null) {
    setState(() {
      _mensagem = 'Operação cancelada. Nenhuma nova imagem obtida.';
    });

    return;
  }

  setState(() {
    _imagem = imagem;
    _mensagem = origem == ImageSource.camera
        ? 'Fotografia capturada com sucesso.'
        : 'Imagem selecionada com sucesso.';
  });
} on PlatformException catch (erro) {
  if (!mounted) return;

  setState(() {
    if (erro.code == 'camera_access_denied' ||
        erro.code == 'photo_access_denied') {
      _mensagem =
          'O acesso não foi autorizado. Verifique as permissões '
          'nas configurações do aplicativo.';
    } else {
      _mensagem =
          'Não foi possível obter a imagem. Verifique a '
          'disponibilidade do recurso e tente novamente.';
    }
  });
} catch (erro) {
  if (!mounted) return;
```

```
      setState(() {
        _mensagem = 'Ocorreu uma falha ao obter a imagem.';
      });
    } finally {
      if (mounted) {
        setState(() {
          _carregando = false;
        });
      }
    }
  }

  void _removerImagem() {
    setState(() {
      _imagem = null;
      _mensagem = 'Imagem removida da apresentação.';
    });
  }

  @override
  Widget build(BuildContext context) {
    final imagem = _imagem;

    return Scaffold(
      appBar: AppBar(title: const Text('Câmera e galeria')),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            ElevatedButton.icon(
              onPressed: _carregando
                  ? null
                  : () => _obterImagem(ImageSource.camera),
              icon: const Icon(Icons.camera_alt),
              label: const Text('Tirar fotografia'),
            ),
            const SizedBox(height: 8),
            ElevatedButton.icon(
              onPressed: _carregando
                  ? null
                  : () => _obterImagem(ImageSource.gallery),
              icon: const Icon(Icons.photo_library),
              label: const Text('Selecionar da galeria'),
            ),
            const SizedBox(height: 16),
            Text(_mensagem),
            const SizedBox(height: 16),
```

```
            if (_carregando) const Center(child: CircularProgressIndicator()),
            const SizedBox(height: 16),
            if (imagem == null)
              Container(
                height: 280,
                alignment: Alignment.center,
                decoration: BoxDecoration(
                  color: Colors.grey.shade200,
                  borderRadius: BorderRadius.circular(12),
                ),
                child: const Text('Nenhuma imagem selecionada'),
              )
            else ...[
              ClipRRect(
                borderRadius: BorderRadius.circular(12),
                child: Image.file(
                  File(imagem.path),
                  height: 280,
                  width: double.infinity,
                  fit: BoxFit.contain,
                  errorBuilder: (context, erro, stackTrace) {
                    return const SizedBox(
                      height: 280,
                      child: Center(
                        child: Text('Não foi possível apresentar a imagem.'),
                      ),
                    );
                  },
                ),
              ),
              const SizedBox(height: 8),
              Text('Arquivo: ${imagem.name}'),
              const SizedBox(height: 8),
              OutlinedButton.icon(
                onPressed: _carregando ? null : _removerImagem,
                icon: const Icon(Icons.clear),
                label: const Text('Remover imagem'),
              ),
            ],
          ],
        ),
      ),
    );
  }
}
```

4.5. Objeto responsável pela seleção

O acesso às operações é realizado por uma instância de `ImagePicker`:

```
final ImagePicker _seletor = ImagePicker();
```

Esse objeto será utilizado tanto para a câmera quanto para a galeria.

A imagem obtida é armazenada em:

```
XFile? _imagem;
```

O tipo `XFile` representa o arquivo retornado pelo pacote. O símbolo `?` permite que a variável contenha `null`, indicando que ainda não existe uma imagem selecionada.

4.6. Captura pela câmera e seleção pela galeria

Os dois botões chamam a mesma função:

```
Future<void> _obterImagem(ImageSource origem)
```

A diferença está no argumento fornecido.

Para a câmera:

```
_obterImagem(ImageSource.camera),
```

Para a galeria:

```
_obterImagem(ImageSource.gallery)
```

Dentro da função, a operação utiliza:

```
final imagem = await _seletor.pickImage(
  source: origem,
  maxWidth: 1200,
  maxHeight: 1200,
  imageQuality: 85,
  requestFullMetadata: false,
);
```

O método `pickImage()` retorna um `Future<XFile?>`. A origem determina qual interface será aberta. Os parâmetros de dimensões e qualidade permitem ajustar a imagem retornada, conforme o formato suportado.

Parâmetro	Finalidade
`source`	Escolher câmera ou galeria.
`maxWidth`	Limitar a largura da imagem retornada.
`maxHeight`	Limitar a altura da imagem retornada.
`imageQuality`	Solicitar qualidade de 0 a 100 para formatos compatíveis.
`requestFullMetadata`	Indicar se o pacote deve buscar metadados completos.

Os valores foram escolhidos para o exemplo. A qualidade 85 não significa que o arquivo terá exatamente 85% do tamanho original.

Utilizamos `requestFullMetadata: false` porque a atividade precisa somente da imagem, sem consultar seus metadados completos. No iOS, essa opção pode evitar solicitações adicionais relacionadas à biblioteca.

4.7. Tratamento do cancelamento

O usuário pode abrir a câmera e retornar sem tirar uma fotografia. Também pode abrir a galeria e desistir da seleção.

Por isso, verificamos o resultado:

```
if (imagem == null) {
  setState(() {
    _mensagem = 'Operação cancelada. Nenhuma nova imagem obtida.';
  });

  return;
}
```

Cancelar uma operação não deve ser apresentado como erro. No exemplo, uma imagem anterior é preservada quando o usuário cancela. Ela somente é substituída quando uma nova imagem é obtida.

4.8. Apresentação da imagem

Quando a operação é concluída, o aplicativo atualiza seu estado:

```
setState(() {
  _imagem = imagem;
  _mensagem = origem == ImageSource.camera
      ? 'Fotografia capturada com sucesso.'
      : 'Imagem selecionada com sucesso.';
});
```

A apresentação utiliza:

```
Image.file(
  File(imagem.path),
  height: 280,
  width: double.infinity,
  fit: BoxFit.contain,
),
```

Nesse trecho:

- `imagem.path` fornece o caminho do arquivo;
- `File()` representa o arquivo local;
- `Image.file()` carrega a imagem;
- `BoxFit.contain` mantém a proporção e apresenta a imagem inteira dentro da área disponível.

O `errorBuilder` oferece uma mensagem alternativa caso o arquivo não possa ser apresentado.

O botão “Remover imagem” executa:

```
setState(() {
  _imagem = null;
  _mensagem = 'Imagem removida da apresentação.';
```

```
});
```

Isso remove a imagem da interface. Não exclui a fotografia original da galeria.

4.9. Permissões e tratamento de falhas

Diferentemente dos exemplos de contatos e localização, não faremos uma solicitação manual de permissão antes de chamar o pacote.

O `image_picker` utiliza o fluxo nativo correspondente à operação. Assim, abrir um seletor de imagens não significa obter acesso irrestrito a toda a biblioteca.

Falhas de integração podem ser retornadas como `PlatformException`, incluindo problemas de autorização ou indisponibilidade do recurso.

No exemplo, o tratamento diferencia alguns códigos de recusa de acesso e apresenta uma mensagem geral para outras falhas.

Durante a operação, os botões ficam desabilitados:

```
onPressed: _carregando ? null : () => _obterImagem(ImageSource.camera),
```

Isso evita iniciar uma nova seleção enquanto a anterior ainda está em andamento.

Após cada operação assíncrona, verificamos `mounted` antes de atualizar a tela:

```
if (!mounted) return;
```

4.10. Recuperação de uma operação interrompida no Android

Enquanto a câmera ou o seletor estão abertos, o Android pode encerrar a atividade do aplicativo para liberar memória. Quando o aplicativo é recriado, o resultado pode não retornar à chamada original.

Para tratar essa situação, o pacote disponibiliza `retrieveLostData()`. O exemplo o executa na inicialização, somente no Android.

```
@override
void initState() {
  super.initState();

  _recuperarImagemPendente();
}
```

A função verifica se existe um resultado pendente:

```
final resposta = await _seletor.retrieveLostData();
if (resposta.isEmpty) return;
```

Se houver arquivos recuperados, o primeiro é apresentado.

Essa recuperação trata uma operação interrompida. Ela não substitui o armazenamento permanente das imagens.

4.11. Selecionar uma imagem não significa armazená-la permanentemente

O exemplo mantém a referência à imagem no estado da tela para apresentação durante o uso.

O arquivo retornado pelo pacote não deve ser tratado como armazenamento permanente. Fotografias capturadas podem estar no cache do aplicativo, e o caminho retornado não deve ser reutilizado entre sessões como garantia de persistência.

Para manter uma imagem vinculada a um cadastro, uma possível sequência seria:

1. Capturar ou selecionar a imagem.
2. Copiar o arquivo para um diretório persistente do aplicativo.
3. Registrar no SQLite o caminho da cópia e os demais dados do cadastro.
4. Recuperar essa referência quando o aplicativo for aberto novamente.

Isso estabelece uma ligação com a aula anterior: o arquivo contém a imagem, enquanto o banco de dados pode armazenar sua referência.

Exercício

Exercício — Consulta aos compromissos do calendário

Desenvolva um aplicativo Flutter que consulte o calendário do dispositivo e liste os compromissos de hoje e dos próximos seis dias, totalizando um período de sete dias.

O aplicativo deverá:

- Solicitar a permissão de acesso ao calendário.
- Disponibilizar um botão “Consultar compromissos”.
- Consultar todos os calendários disponíveis no dispositivo.
- Listar os compromissos em ordem cronológica.
- Exibir, para cada compromisso: título, calendário, data e horário de início e fim, local e descrição.
- Identificar os compromissos que ocupam o dia inteiro.
- Exibir um indicador de carregamento durante a consulta.
- Informar quando não houver compromissos, quando o acesso for negado ou quando ocorrer uma falha na consulta.

Utilize o pacote `device_calendar`, e configure as permissões necessárias no Android.